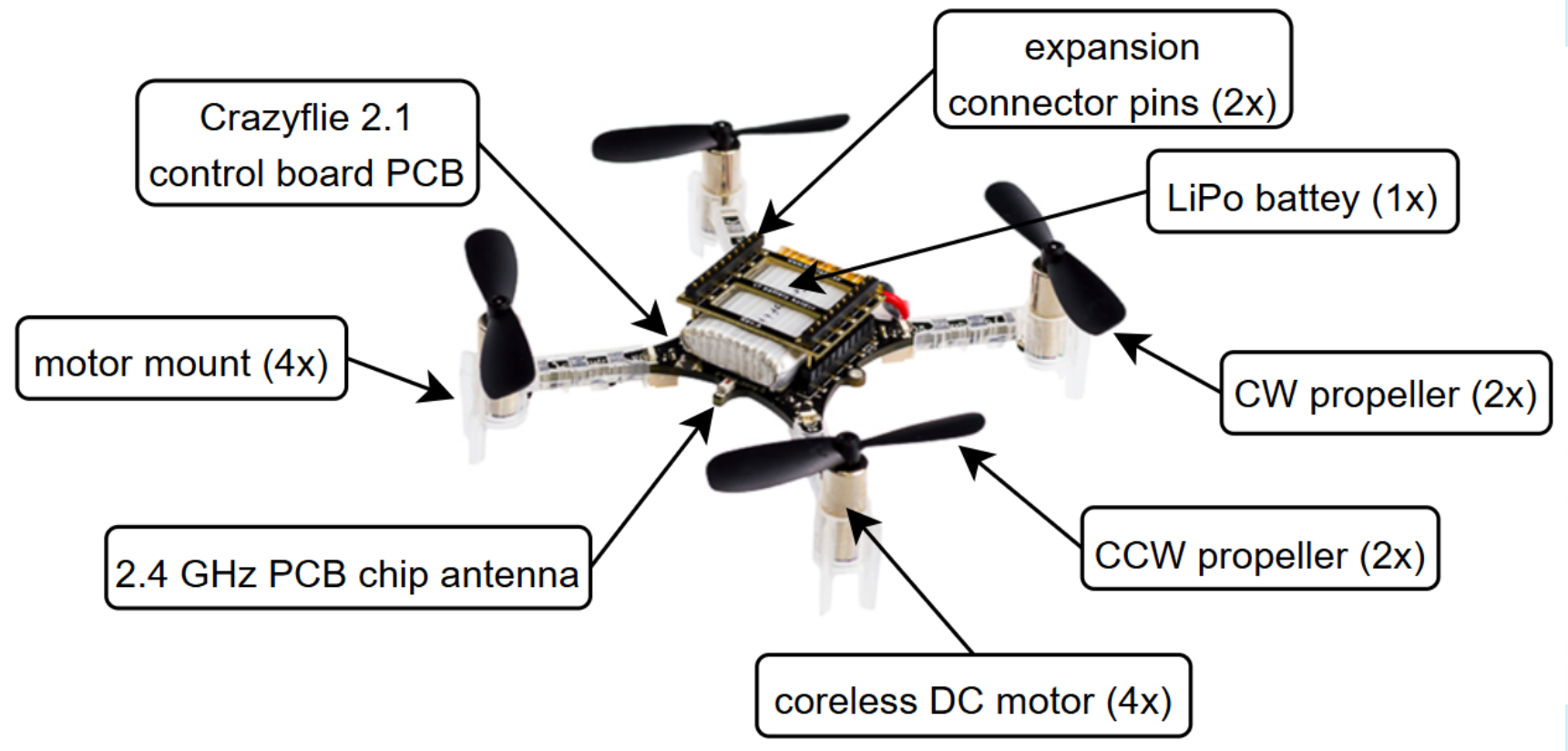
Ρομποτικά Συστήματα ΙΙ

CRAZYFLIES – GETTING STARTED

Crazyflie drone parts



Expansion decks:

lighthouse deck

optic flow deck

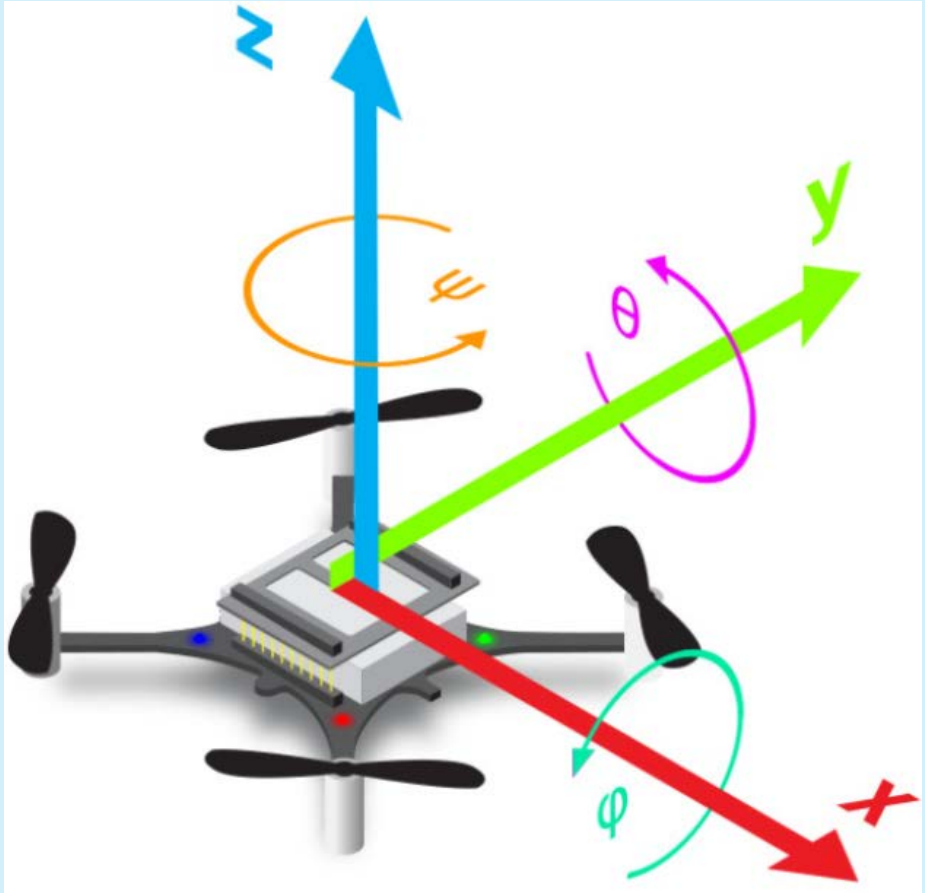
Constructing the crazyflie drone

Go to this link and follow the instructions carefully :

https://www.bitcraze.io/documentation/tutorials/getting-started-with-crazyflie-2-x/

At the end of the construction process, the crazyflie drone should look like in the picture below:

Crazyflie coordinates system



The global coordinate system follows the ENU (EAST NORTH UP) convention, and the local body fixed frame is defined similarly.
The body frame's origin is attached at the crazyflie's center of gravity, as shown in the picture. The crazyflie points towards the x-axis, and the z-axis points up.
The roll and yaw angles are defined in a clockwise convention, but the pitch angle is defined counter-clockwise.

Crazyflie Dynamic Model

We assume that the state of the quadrotor is given by: $\boldsymbol{x} = [\boldsymbol{r_w} \quad \boldsymbol{R_{wb}} \quad \boldsymbol{v_b} \quad \boldsymbol{\omega_b}]$ or $\boldsymbol{x} = [\boldsymbol{r_w} \quad \boldsymbol{q_{wb}} \quad \boldsymbol{v_b} \quad \boldsymbol{\omega_b}]$, where:

- $\boldsymbol{r_w} \in \mathbb{R}^{3\times1}$ is the position in the world frame
- $\boldsymbol{R_{wb}} \in \mathbb{R}^{3\times3}$/ $\boldsymbol{q_{wb}} \in \mathbb{R}^{4\times1}$ is the orientation of the body frame w.r.t. to the world frame
- $\boldsymbol{v_b} \in \mathbb{R}^{3\times1}$ is the linear velocity in the body frame
- $\boldsymbol{\omega_b} \in \mathbb{R}^{3\times1}$ is the angular velocity in the body frame

And the dynamics are written like this:

$$\begin{aligned} \dot{\boldsymbol{r}}_{\boldsymbol{w}} &= \boldsymbol{R_{wb}}\boldsymbol{v_b} \\ \dot{\boldsymbol{v}}_{\boldsymbol{b}} &= \frac{1}{m}\boldsymbol{F_b} - \boldsymbol{\omega_b} \times \boldsymbol{v_b} \\ \dot{\boldsymbol{\omega}}_{\boldsymbol{b}} &= \boldsymbol{I}^{-1}[\boldsymbol{T_b} - \boldsymbol{\omega_b} \times (\boldsymbol{I}\boldsymbol{\omega_b})] \end{aligned} \quad , \quad \boldsymbol{F_b} = \boldsymbol{R}_{\boldsymbol{wb}}^{\top} \cdot \begin{bmatrix} \boldsymbol{0} \\ \boldsymbol{0} \\ -\boldsymbol{mg} \end{bmatrix} + \begin{bmatrix} \boldsymbol{0} \\ \boldsymbol{0} \\ \sum_{i=1}^{4} \boldsymbol{F_i} \end{bmatrix}, \quad \boldsymbol{T_b} = \begin{bmatrix} \boldsymbol{l} \cdot [(\boldsymbol{F_1} - \boldsymbol{F_3})\,\boldsymbol{cos}\,\boldsymbol{\theta} - (\boldsymbol{F_4} - \boldsymbol{F_2})\,\boldsymbol{sin}\,\boldsymbol{\theta}] \\ \boldsymbol{l} \cdot [(\boldsymbol{F_1} - \boldsymbol{F_3})\,\boldsymbol{sin}\,\boldsymbol{\theta} + (\boldsymbol{F_4} - \boldsymbol{F_2})\,\boldsymbol{cos}\,\boldsymbol{\theta}] \\ \boldsymbol{K_m}(\boldsymbol{F_2} + \boldsymbol{F_4} - \boldsymbol{F_1} - \boldsymbol{F_3}) \end{bmatrix}$$

Where: $\boldsymbol{F_i} = \boldsymbol{K_f} \cdot \boldsymbol{\omega}_{\boldsymbol{r,i}}^{\boldsymbol{2}}$ and $\boldsymbol{T_i} = \boldsymbol{K_m} \cdot \boldsymbol{F_i} = \boldsymbol{K_m} \cdot \boldsymbol{K_f} \cdot \boldsymbol{\omega}_{\boldsymbol{r,i}}^{\boldsymbol{2}} = \boldsymbol{K_t} \cdot \boldsymbol{\omega}_{\boldsymbol{r,i}}^{\boldsymbol{2}}$ are the thrust (in N) and torque (in Nm) respectively, provided by one rotor rotating with $\omega_{r,i}$ angular speed (in rad/sec). (Note: for the various parameters shown, you should check the next slide).

For finding the orientation of the crazyflie at the next time step dt, you can manipulate the quaternion q_{wb} or use the rotation group $SO(3)$ and the relation $\boldsymbol{R}'_{\boldsymbol{wb}} = \boldsymbol{R_{wb}} \oplus (\boldsymbol{\omega_b}\boldsymbol{dt})$.

Crazyflie Dynamic Model

As control inputs of the crazyflie's quadrotor system you may assume the thrusts provided by each rotor or their angular speeds, meaning that:

$$\boldsymbol{u} = [\boldsymbol{F_1} \quad \boldsymbol{F_2} \quad \boldsymbol{F_3} \quad \boldsymbol{F_4}] \quad \text{or} \quad \boldsymbol{u} = [\boldsymbol{\omega_{r,1}} \quad \boldsymbol{\omega_{r,2}} \quad \boldsymbol{\omega_{r,3}} \quad \boldsymbol{\omega_{r,4}}]$$

Also, for the model parameters, you may use these values:

- $m = 0.033\ kg$ (mass)
- $l = 0.046\ m$ (arm length)
- $K_f = 2.25 \cdot 10^{-8}\ \frac{N}{(rad/sec)^2}$ (thrust's coefficient for one rotor)
- $K_t = 1.34 \cdot 10^{-10}\ \frac{Nm}{(rad/sec)^2}$ (torque's coefficient for one rotor)
- $\theta = -\frac{3\pi}{4}\ rad$ (it shows the orientation of the body frame on xy plane, when looking from above)

For the dynamics of the crazyflie and some model parameters and relations, check also:

- https://lup.lub.lu.se/luur/download?func=downloadFile&recordOId=8905295&fileOId=8905299 (Appendix A)
- https://www.research-collection.ethz.ch/handle/20.500.11850/214143 (Chapter 3)

Crazyflie parameters and log variables

The Crazyflie firmware allows the creation of custom parameters and logging variables. Parameters can be modified via the cfclient or the Python API, enabling changes to system behavior without reloading the firmware. Logging variables, on the other hand, are read-only and can be monitored—also through cfclient or the API—to observe the evolution of specific values over time. In cfclient, the dedicated tabs for these functionalities are “Parameters” and “Log TOC”, where the Table of Contents (TOC) for each can be explored. Parameters and log variables follow the naming convention “group.name”.

Prerequisites – Conda environment

- Python 3.10+
- Create conda environment (this is optional).

conda create –n crazyflies python=3.10

- Activate the newly created conda environment.

conda activate crazyflies

The conda environment "*crazyflies*" is now set as our working place.

If you want to get out of the conda environment you can run "conda deactivate".

Prerequisites – Python API

- Install the crazyflies Python API, that implements a python interface to code.

git clone https://github.com/bitcraze/crazyflie-lib-python.git

cd crazyflie-lib-python

pip3 install -e .

or

pip3 install cflib

Prerequisites – Python crazyflies client

- Install the python client.

git clone https://github.com/bitcraze/crazyflie-clients-python.git

cd crazyflie-clients-python

pip3 install -e .

or

pip3 install cfclient

Prerequisites – Flash crazyradio dongle PA-firmware

- Flash the right PA-firmware into the crazyradio dongle. It's important for the crazyradio dongle to be in PA (Power Amplifier) mode, with full amplifier and LNA (Low Noise Amplifier) functionality enabled.
- Screw the antenna on to the connector of the USB dongle, and then press and hold the button while inserting the dongle into a USB port. If the LED is pulsating with a red light, this indicates that the Crazyradio 2.0 is in bootloader mode.
- Go to https://github.com/bitcraze/crazyradio2-firmware/releases/, download the file named "crazyradio2-CRPA-emulation-[version].uf2", and then, in your file browser, drag and drop the downloaded file to the Crazyradio 2.0 drive.
- After the installation, the LED will light up briefly in white, and this will happen every time you re-plug the dongle. The Crazyradio 2.0 is now behaving like a Crazyradio PA.

Prerequisites – Install drivers

- Depending on which operating system you use, you will have to install drivers or do some configuration to communicate with the Crazyradio 2.0.
- If you are on Windows, you can follow the instructions here:

https://www.bitcraze.io/documentation/repository/crazyradio-firmware/master/building/usbwindows/

- If you are on Linux, you need to setup the UDEV permissions for the USB port. To do so, follow the instructions here:

https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/installation/usb_permissions/

Prerequisites – Load the newest firmware in the crazyflie

- Open the cfclient.
- Connect to the Crazyflie with the Crazyradio.
- Go to **Connect** -> **Bootloader**.
- In the “Firmware source” window choose “cf2” as the Platform Filter and the newest firmware release from the “Available downloads”.
- Press “**Program**” and wait until the loading is completed. The Crazyflie may restart one or more times during the programming process.

Flashing new firmware

- Install some necessary packages and clone the crazyflie firmware repository.

sudo apt-get install make gcc-arm-none-eabi

git clone --recursive https://github.com/bitcraze/crazyflie-firmware.git

- Start building (first, make sure that you have swig installed on your system, and then run the commands below).

cd crazyflie-firmware & **make cf2_defconfig** & **make** or **make -j$(nproc)**

make bindings_python & **cd build** & **python3 setup.py install --user**

- Build new firmware for flashing (run this every time you want to flash a new firmware).

make –j$(nproc) or **make –j8** (if already there is a desired config to rebuild)

- Start the Crazyflie in bootloader mode by pressing the power button for 3 seconds. Both the blue LEDs will start blinking. Then, connect the crazyradio dongle into a USB port and type the command below.

CLOAD_CMDS="-w [CRAZYFLIE_URI]" make cload or **make cload**

References

1. **Εισαγωγική γενική παρουσίαση της Bitcraze:**
   https://www.bitcraze.io/about/events/documents/EPFL_Bitcraze_lecture_2021.pdf

2. **For the dynamic model of the crazyflie:**
   https://www.bitcraze.io/2018/11/demystifying-drone-dynamics
   https://lup.lub.lu.se/luur/download?func=downloadFile&recordOId=8905295&fileOId=8905299
   (look at Appendix A)
   https://www.research-collection.ethz.ch/handle/20.500.11850/214143 (look at Chapter 3)

3. **Lighthouse positioning system:**
   https://www.bitcraze.io/documentation/system/positioning/ligthouse-positioning-system/
   https://www.bitcraze.io/documentation/tutorials/getting-started-with-lighthouse/

4. **Implementing your own controller or estimator in the firmware:**
   https://www.bitcraze.io/2023/02/adding-an-estimator-or-controller/